Instagram-Style Social App

MVP Technical Specification — Initial Release for 10 Users

Objective: build a small, production-shaped social photo application for an initial private pilot of 10 users. The architecture is intentionally simple, but the data model and media pipeline should not create avoidable scaling problems later.

Area	Decision
Pilot	10 users
Architecture	Modular monolith
Backend	Node.js + TypeScript
Mobile	React Native or Flutter
API	REST over HTTPS
Database	PostgreSQL
Media	S3-compatible object storage
Cache/jobs	Redis when needed
Feed	Chronological, following-only
Initial goal	Validate the social loop, not maximize infrastructure

Core loop: register → profile → follow → publish photo → view feed → like/comment → notification. Defer Stories, Reels, DMs, recommendations, ads, live video and creator analytics.

1. MVP Scope

Included

- Registration, login, logout and session refresh.
- Profile: username, avatar, bio, follower/following counts.
- User search by username.
- Follow/unfollow.
- Create and delete photo posts with captions.
- Chronological feed of followed accounts.
- Profile post grid.
- Like/unlike.
- Comments: create, list, delete own comment.
- Notifications for follows, likes and comments.
- Block and report.
- Basic admin moderation.
- Account/content deletion.

Not included

- Stories, Reels/video, live streaming, DMs.
- Recommendation/ML ranking.
- Ads, payments, shopping, creator monetization.
- Microservices, Kubernetes, Kafka, multi-region deployment.
- Advanced search, threaded comments, trending hashtags.

2. User Flows

Registration

1. Register with email, username and password.
2. Validate uniqueness and password policy.
3. Hash password with Argon2id (or equivalent).
4. Create user.
5. Issue short-lived access token + refresh token.
6. Complete profile.
7. Search/follow another user.

Photo publishing

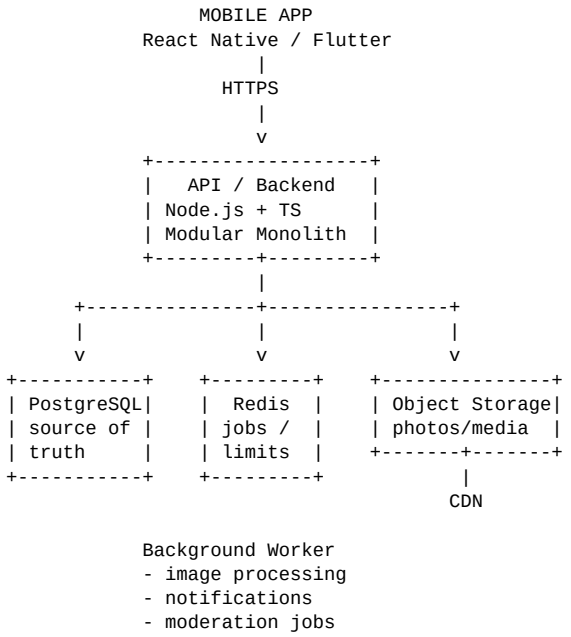
```
Mobile -> POST /posts/upload-url
API -> presigned upload URL + object key
Mobile -> direct upload to object storage
Mobile -> POST /posts {media_key, caption}
API -> validates object ownership -> creates post
Worker -> creates optimized variants
Feed -> serves optimized image
```

Feed

```
GET /feed?limit=20&cursor=<cursor>

Find followed users -> fetch published posts ->
ORDER BY created_at DESC, id DESC -> return 20 + next_cursor
```

3. System Architecture



For 10 users, one backend instance + one managed PostgreSQL database + object storage is enough. Redis can be introduced when background jobs or rate limiting require it. Do not use microservices simply because the product resembles Instagram.

4. Database Model

PostgreSQL is the source of truth. Use UUIDs (or another globally unique ID), UTC timestamps, foreign keys, unique constraints and indexes. Do not store image binaries in PostgreSQL.

```
users 1 ---- * posts
users 1 ---- * comments
users * ---- * users       (follows)
users * ---- * posts       (likes)
users 1 ---- * notifications
users * ---- * users       (blocks)
users 1 ---- * reports
posts 1 ---- * comments
posts 1 ---- * likes
posts 1 ---- * reports
```

Table	Purpose	Important columns
users	Accounts/profiles	id, username, email, password_hash, bio, avatar_key
refresh_tokens	Sessions	id, user_id, token_hash, expires_at, revoked_at
follows	Social graph	follower_id, following_id, created_at
posts	Photo posts	id, user_id, caption, status, created_at
post_media	Image variants	id, post_id, variant, object_key, width, height
likes	Likes	user_id, post_id, created_at
comments	Comments	id, post_id, user_id, body, created_at, deleted_at
notifications	Activity	id, user_id, actor_id, type, post_id, read_at
blocks	Blocking	blocker_id, blocked_id, created_at
reports	Moderation	id, reporter_id, target_type, target_id, reason, status

5. Full SQL Schema

users

```
CREATE TABLE users (
```

```
  id UUID PRIMARY KEY,
  username VARCHAR(30) NOT NULL UNIQUE,
  email VARCHAR(320) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  bio VARCHAR(160),
  avatar_key TEXT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_users_username_lower ON users (LOWER(username));
```

refresh_tokens

```
CREATE TABLE refresh_tokens (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash TEXT NOT NULL UNIQUE,
  expires_at TIMESTAMPTZ NOT NULL,
  revoked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_refresh_tokens_user ON refresh_tokens(user_id);
```

follows

```
CREATE TABLE follows (
  follower_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  following_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (follower_id, following_id),
  CHECK (follower_id <> following_id)
);
CREATE INDEX idx_follows_following ON follows(following_id, created_at DESC);
```

posts

```
CREATE TABLE posts (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  caption VARCHAR(2200),
  status VARCHAR(20) NOT NULL DEFAULT 'PUBLISHED',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CHECK (status IN ('PROCESSING','PUBLISHED','HIDDEN','DELETED'))
);
CREATE INDEX idx_posts_user_created ON posts(user_id, created_at DESC, id DESC);
```

post_media

```
CREATE TABLE post_media (
  id UUID PRIMARY KEY,
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  variant VARCHAR(20) NOT NULL,
  object_key TEXT NOT NULL,
  width INTEGER,
  height INTEGER,
  mime_type VARCHAR(100),
  byte_size BIGINT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (post_id, variant)
);
CREATE INDEX idx_post_media_post ON post_media(post_id);
```

likes

```
CREATE TABLE likes (
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, post_id)
);
CREATE INDEX idx_likes_post ON likes(post_id, created_at DESC);
```

comments

```
CREATE TABLE comments (
  id UUID PRIMARY KEY,
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  body VARCHAR(1000) NOT NULL,
```

```
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);
CREATE INDEX idx_comments_post_created ON comments(post_id, created_at ASC, id ASC);
```

notifications

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  actor_id UUID REFERENCES users(id) ON DELETE SET NULL,
  type VARCHAR(20) NOT NULL,
  post_id UUID REFERENCES posts(id) ON DELETE CASCADE,
  read_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CHECK (type IN ('FOLLOW','LIKE','COMMENT'))
);
CREATE INDEX idx_notifications_user_created ON notifications(user_id, created_at DESC);
```

blocks

```
CREATE TABLE blocks (
  blocker_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  blocked_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (blocker_id, blocked_id),
  CHECK (blocker_id <> blocked_id)
);
CREATE INDEX idx_blocks_blocked ON blocks(blocked_id);
```

reports

```
CREATE TABLE reports (
  id UUID PRIMARY KEY,
  reporter_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  target_type VARCHAR(20) NOT NULL,
  target_id UUID NOT NULL,
  reason VARCHAR(50) NOT NULL,
  details VARCHAR(1000),
  status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  resolved_at TIMESTAMPTZ,
  CHECK (target_type IN ('USER','POST','COMMENT')),
  CHECK (status IN ('OPEN','REVIEWING','RESOLVED','DISMISSED'))
);
CREATE INDEX idx_reports_status_created ON reports(status, created_at ASC);
```

6. API Specification

Method	Endpoint	Purpose
POST	/auth/register	Create account
POST	/auth/login	Login
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Revoke session
GET	/me	Current user
PATCH	/me	Update profile
POST	/me/avatar/upload-url	Avatar upload URL
GET	/users/:username	Profile
GET	/users/:username/posts	Profile posts
POST	/users/:id/follow	Follow
DELETE	/users/:id/follow	Unfollow
POST	/posts/upload-url	Post upload URL
POST	/posts	Create post
GET	/posts/:id	Get post
DELETE	/posts/:id	Delete post

Method	Endpoint	Purpose
GET	/feed	Home feed
POST	/posts/:id/like	Like
DELETE	/posts/:id/like	Unlike
GET	/posts/:id/comments	List comments
POST	/posts/:id/comments	Add comment
DELETE	/comments/:id	Delete comment
GET	/notifications	Notifications
POST	/notifications/:id/read	Mark read
POST	/users/:id/block	Block
DELETE	/users/:id/block	Unblock
POST	/reports	Report

Create post example

```
POST /posts
{
  "media_key": "users/<user-id>/posts/<post-id>/original.jpg",
  "caption": "My first post"
}

Response:
{
  "id": "<post-id>",
  "status": "PROCESSING",
  "caption": "My first post",
  "created_at": "2026-10-03T16:00:00Z"
}
```

Feed response example

```
GET /feed?limit=20&cursor=...

{
  "items": [{
    "id": "<post-id>",
    "user": {"id": "<user-id>", "username": "alice", "avatar_url": "..."},
    "media": {"feed_url": "...", "width": 1080, "height": 1350},
    "caption": "Hello world",
    "like_count": 3,
    "comment_count": 1,
    "liked_by_me": true,
    "created_at": "2026-10-03T16:00:00Z"
  }],
  "next_cursor": "..."
}
```

7. Feed Query

```
SELECT p.id, p.user_id, p.caption, p.created_at
FROM posts p
JOIN follows f ON f.following_id = p.user_id
WHERE f.follower_id = :viewer_id
  AND p.status = 'PUBLISHED'
  AND (p.created_at, p.id) < (:cursor_time, :cursor_id)
ORDER BY p.created_at DESC, p.id DESC
LIMIT 20;
```

Use cursor pagination rather than page numbers. At 10 users, fan-out-on-read is sufficient. Later optimize indexes, caching, counters or fan-out based on measured traffic.

8. Media Pipeline

- Store media in object storage, not PostgreSQL.
- API issues a short-lived presigned upload URL.
- Mobile uploads directly to storage.

- API verifies object ownership before creating a post.
- Background worker creates optimized variants.
- Recommended variants: original, feed, thumbnail.
- Serve feed-sized images instead of originals.
- Reject invalid, oversized or malicious files.

Object-key convention

```
users/{user_id}/avatars/{uuid}.jpg
users/{user_id}/posts/{post_id}/original.jpg
users/{user_id}/posts/{post_id}/feed.jpg
users/{user_id}/posts/{post_id}/thumb.jpg
```

9. Mobile Screens

Screen	Responsibility
Welcome/Login	Register, login, logout
Home	Feed, refresh, pagination
Create Post	Select photo, preview, caption, publish
Post Detail	Photo, caption, likes, comments
Search	Username search
Profile	Bio, counts, grid, follow
Edit Profile	Avatar/bio/profile fields
Activity	Notifications
Settings	Blocked users, logout, delete account
Report	Reason + submit

10. Backend Code Structure

```
src/
  app/
    routes/
    middleware/
    errors/
  modules/
    auth/
    users/
    follows/
    posts/
    feed/
    likes/
    comments/
    notifications/
    moderation/
  infrastructure/
    database/
    storage/
    redis/
    queue/
  workers/
    image-processing.worker.ts
    notification.worker.ts
  shared/
    config/
    logging/
    validation/
    security/
```

11. Security & Abuse

- HTTPS outside local development.
- Argon2id password hashing.
- Short-lived access tokens and revocable refresh tokens.
- Server-side authorization for every protected action.
- Rate-limit login, registration, uploads, follows, likes and comments.
- Validate all input and upload content server-side.
- Use foreign keys, unique constraints and parameterized DB access.
- Keep secrets out of source control.
- Never log passwords or raw authentication tokens.
- Automatic backups and tested restoration.
- Block/report/admin moderation before the pilot expands.

12. Development Plan for 10 Users

Phase	Work	Exit condition
1	Repo, backend, mobile shell, PostgreSQL, migrations	Everything runs locally
2	Auth + profiles	Register/login/edit profile
3	Follow + search	Users can follow each other
4	Storage + posts	Photo can be published/deleted
5	Feed + profile grid	Followed posts appear
6	Likes + comments	Engagement works
7	Notifications	Activity visible
8	Block/report/admin	Basic abuse workflow
9	Tests/monitoring/backups	Restore and failure visibility tested
10	Private pilot	10 users complete core loop

13. Testing Strategy

- Unit tests for auth, authorization, follow rules and ownership.
- Integration tests against test PostgreSQL.
- API tests for authenticated endpoints.
- Mobile tests for login, feed, upload, like/comment and logout.
- Upload tests for invalid files and unauthorized object keys.
- Cross-user authorization tests.
- Basic load test before increasing the pilot.

14. Observability & Product Metrics

- API latency, error rate and slow queries.
- Image upload/background-job failures.
- Database health and backup status.
- Registered users and DAU.
- Posts, likes, comments and follows per day.
- D1/D7/D30 retention.
- Percentage of new users who follow someone.
- Percentage of new users who publish a first post.
- Reports and moderation actions.

15. Deployment Checklist

- Production PostgreSQL + automated backups.
- HTTPS backend.
- Secure environment variables/secrets.
- Object storage configured.
- Migrations integrated into deployment.
- Error monitoring and structured logs.
- Rate limits enabled.
- Admin access protected.
- Deletion flows tested.
- Backup restoration tested.

16. What Not to Build Yet

Feature	Reason to defer
Reels/video	Media, bandwidth, processing and moderation complexity
Stories	Expiration/visibility complexity
DMs	Delivery, state and abuse controls
Recommendations	Needs behavioral data and ranking experiments
Microservices	Operational overhead
Kubernetes	No orchestration need
Kafka	Jobs/events are small at this scale
Search engine	PostgreSQL search is sufficient
Feed fan-out	Read-time feed is sufficient

17. MVP Acceptance Criteria

- New user can register and log in.
- Users can find and follow each other.
- Photo uploads go directly to object storage.
- Published posts appear in followers’ feeds.
- Like/unlike works.
- Comments can be created and deleted by their owner.
- Follow/like/comment create notifications.
- Blocking changes visibility/interactions as intended.
- Reports appear in admin moderation.
- Admin can hide content or suspend an account.
- Users can delete content and initiate account deletion.
- Database restoration has been tested.
- Errors and failed jobs are observable.

18. Scaling Path: 10 → 10,000

Stage	Recommended architecture
10 users	1 API + PostgreSQL + object storage; Redis optional
100–1,000	Managed DB + Redis for jobs/rate limits + CDN + worker
1,000–10,000	Multiple API instances, pooling, workers, monitoring, query/feed optimization

Stage	Recommended architecture
10,000+	Evaluate read replicas, feed caching/fan-out, dedicated search, queue infrastructure, service extraction

Important: 10 registered users, 10 daily active users, and 10,000 concurrent users are different workloads. Scale from observed traffic rather than registered-user count alone.

19. Migration Order

```
001_create_users
002_create_refresh_tokens
003_create_follows
004_create_posts
005_create_post_media
006_create_likes
007_create_comments
008_create_notifications
009_create_blocks
010_create_reports
011_add_indexes
012_add_admin/audit fields if required
```

20. Final Recommendation

Start with a modular monolith. PostgreSQL stores durable application data. Object storage holds photos. Direct uploads keep the API lightweight. The feed is chronological and following-only. Cursor pagination is used from day one. Redis is added for concrete needs, not because it is fashionable.

The 10-user pilot should answer: Can users complete the social loop? Do they return? What behavior creates value? Once those questions are known, scale the infrastructure according to real measurements and product demand.